INDIVIDUAL LABORATORY REPORT

DevOps Lab
Project Report

Linux | Git & GitHub | Docker | Jenkins

STUDENT

Alisher Panabaev
Group It2-2308 | Student ID 37332

PROJECT STATUS

Jenkins Pipeline build #2 completed with Finished: SUCCESS. All five stages passed; the application, Docker image, and running container were verified.

19 October 2026

Repository: github.com/alisherqa06/panabaev-alisher-devops

1. Student Information

This report documents the existing Linux project and the implementation completed in the repository, including the actual successful local Jenkins Pipeline build.

Name	Alisher
Surname	Panabaev
Group	It2-2308
Student ID	37332
Course	DevOps
Linux project	`/home/acer/devops/panabaev_alisher`

2. Linux

The existing project runs in Ubuntu 26.04.1 LTS under WSL 2. The WSL status check showed Ubuntu running with version 2 and the Docker Desktop internal distribution running with version 2. The project directory contains app, scripts, config, logs, and backup directories.

The Bash script `scripts/Panabaev_Alisher_system.sh` resolves the student information file relative to its own location. A syntax check and actual run succeeded. Its output included the student record, Linux user acer, hostname KOMPUTER, date, OS release, disk use, memory use, and the successful existence check for the student information file.

```
bash -n scripts/Panabaev_Alisher_system.sh
./scripts/Panabaev_Alisher_system.sh
stat -c '%a %n' private.txt public.txt scripts/Panabaev_Alisher_system.sh
```

Verified permissions were 600 for private.txt, 644 for public.txt, and 755 for the Bash script. The log file had two ERROR entries and two WARNING entries; it remains excluded from Git by the *.log rule. The ignored backup directory and .env rule were also checked.

```
find logs -maxdepth 1 -type f -name '*.log'
grep -c ERROR logs/application.log
grep -c WARNING logs/application.log
head -n 2 logs/application.log
tail -n 2 logs/application.log
ps aux | head
```

Linux provides the shell and standard system tools used in the lab for file operations, permissions, process inspection, system information, and log searches.

3. Git & GitHub

The existing Git history was preserved. The original commits and implementation commits remain in order. The latest fix, `d60133dab3eda0383b184feeba0f14f5ae586972` (Fix Jenkins HTTP check through Docker host bridge), is on both existing remote branches; main and development are aligned.

```
git status --short --branch
git log --oneline --decorate -8
git ls-remote origin refs/heads/main refs/heads/development
```

Git records and identifies source revisions. GitHub hosts the remote repository and makes a selected revision available to Jenkins. Jenkins uses a committed revision so the build can be traced and repeated from the same

source state.

Repository: https://github.com/alisherqa06/panabaev-alisher-devops

4. Docker

Docker Desktop is the Linux container engine for the Windows 11 and WSL 2 setup. In Ubuntu, Docker CLI 29.8.2 and Docker Compose 5.6.0 were available, `docker info` reached the Desktop engine, and the requested hello-world image ran successfully.

The app is a standard-library Python HTTP server. It displays the four student values from environment variables, falls back to the personalized defaults when values are missing or empty, serves the page on port 8080, and provides `/healthz` for readiness checks. HTML output escapes student values.

The Dockerfile uses FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD. It creates a non-root app user. The image name is `panabaev-alisher-devops`; the required container name is `panabaev-alisher-container`. The image is the reusable package, while the container is an instance running its process and environment.

```
docker build -t panabaev-alisher-devops .
docker run -d --name panabaev-alisher-container \
  -p 8080:8080 -e STUDENT_NAME=Alisher \
  -e STUDENT_SURNAME=Panabaev -e STUDENT_GROUP=It2-2308 \
  -e STUDENT_ID=37332 panabaev-alisher-devops
python3 scripts/check_http.py http://127.0.0.1:8080
```

Four unit tests passed: personalized defaults, environment overrides, HTML escaping, and the live health/student-page endpoints. The HTTP smoke check confirmed the expected name, surname, group, and ID. The container logs showed those values, docker exec succeeded, and stop/start and removal were checked.

Docker Compose also completed up --build -d, ps, logs, and down. The HTTP check passed while the Compose container was running. The same image and container names are configured in docker-compose.yml.

```
docker compose up --build -d
docker compose ps
docker compose logs
docker compose down
```

5. Jenkins

A custom Jenkins LTS controller image runs in Docker Desktop on port 8081. The setup wizard, recommended plugins, and local administrator setup completed. Docker CLI 29.8.2 inside Jenkins reaches the same Docker Desktop engine through the mounted WSL socket.

The exact Pipeline job `Panabaev_Alisher_DevOps` is configured with Pipeline script from SCM, Git, the public GitHub repository, branch `*/main`, and script path `Jenkinsfile`. Jenkins build #1 exposed that localhost inside the Jenkins container referred to Jenkins itself. The health check was corrected to use Docker Desktop's `host.docker.internal` bridge; commit `d60133dab3eda0383b184feeba0f14f5ae586972` was pushed to both existing branches.

Jenkins build #2 checked out that commit and completed all five stages in 9.1 seconds. The Console Output ends with the actual `Finished: SUCCESS`. The build ran four unit tests, built `panabaev-alisher-devops`, started `panabaev-alisher-container`, and passed the HTTP smoke check with the expected student values.

The committed Jenkinsfile defines these five stages in order:

Checkout	Checks out the configured SCM source, prints student environment values, and records the source commit.
Build	Compiles the Python app and HTTP smoke-check script.

Test	Runs the standard-library unittest suite.
Docker Build	Builds panabaev-alisher-devops.
Docker Run	Removes an old named container if present, starts the app with the required values, waits for HTTP readiness, checks the page, and prints logs.

The first build failed at the HTTP smoke check, which helped identify the incorrect loopback address. The fix was committed and pushed before the successful second build. Pipeline commands are fail-fast, so a failed required command stops later stages and marks the build unsuccessful.

6. Integrated DevOps Workflow

The intended flow is Linux -> Git -> GitHub -> Jenkins -> Build -> Test -> Docker Build -> Docker Run -> SUCCESS.

Linux	Provides the shell and system utilities used to develop and inspect the project.
Git	Tracks changes, commits, branches, and the exact source revision.
GitHub	Stores the remote repository and lets Jenkins fetch the committed source.
Jenkins checkout	Uses the configured GitHub branch and records the source commit used by a build.
Docker image	Packages the application and its runtime into a reusable artifact.
Docker container	Runs an instance of that image with student environment values and published port 8080.
Pipeline failure	Stops subsequent stages and records the failing command output; it cannot be called successful.

The complete workflow now ran through Jenkins: Checkout -> Build -> Test -> Docker Build -> Docker Run. Build #2 reports Finished: SUCCESS, and the same Docker image and named running container are visible through the shared Docker Desktop engine.

7. Screenshots

The course requests six genuine screenshots. The application screenshot supplied by the student is included on the next page. The remaining five live screens still need to be captured:

- Jenkins Dashboard and the Panabaev_Alisher_DevOps job.

- Build #2 Stages view with Checkout, Build, Test, Docker Build, and Docker Run all green.

- Build #2 Console Output showing the student values, source commit, HTTP smoke check, and Finished: SUCCESS.

- Docker image panabaev-alisher-devops and running container panabaev-alisher-container.

- GitHub repository page showing the latest commit and the aligned main and development branches.

Keep the screenshots free of passwords, tokens, and other secrets.

Screenshot evidence: running application

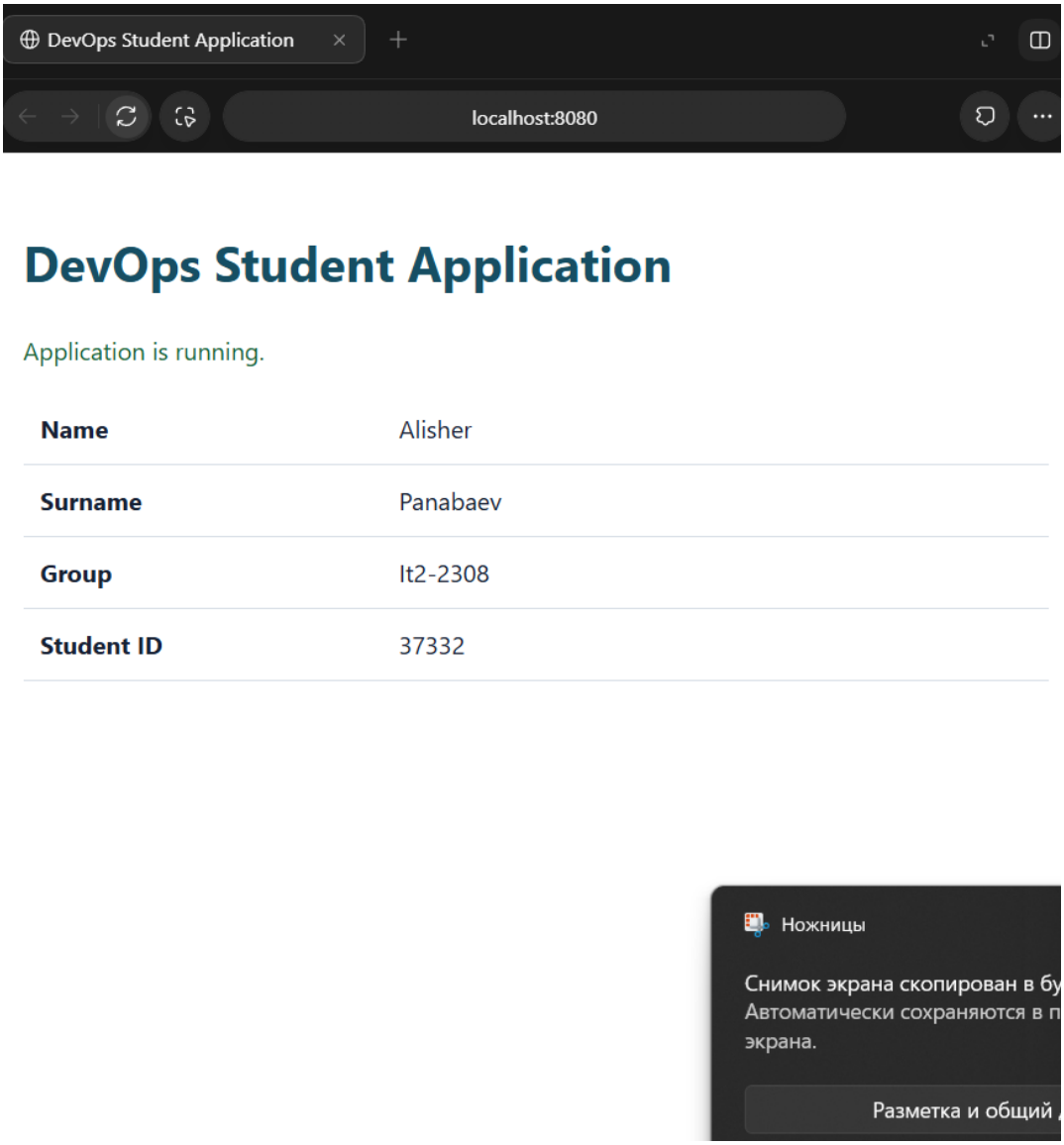


Figure 1. Student supplied screen capture of the live application at http://localhost:8080. The page displays the expected student name, group, and ID.

8. Conclusion

The original Linux repository and commit history were preserved. The student HTTP application, four unit tests, Docker image, named container, Compose configuration, Jenkinsfile, Jenkins controller, README, video script, and hand-in checklist are present in the repository. Jenkins build #2 completed all five stages and reports Finished: SUCCESS.

The project work and Jenkins pipeline are verified. The student must still record their own 10-20 minute voice demonstration and save the screenshots needed by the course hand-in.

Four voice sample recordings totaling about 50 seconds were provided and kept local. No suitable local voice-cloning model was installed. Windows listed standard Russian SAPI voices, but a local synthesis attempt failed, so no usable narration audio was created. No local screen recorder or MP4 encoder was available; no video was produced. The Russian subtitle file is a timing draft based on the script, estimated at about 9 minutes, and has not been synchronized against audio or footage. The course may not accept synthetic narration in place of the student's own voice.

Video demonstration: record one continuous 10-20 minute screen-and-voice walkthrough as the student, following `docs/video-demo-script-ru.md`.

References

Docker Docs, Docker Desktop WSL 2 backend: https://docs.docker.com/desktop/features/wsl

Jenkins Handbook, Getting started with Pipeline: https://www.jenkins.io/doc/book/pipeline/getting-started/

Jenkins Handbook, Docker: https://www.jenkins.io/doc/book/installing/docker/